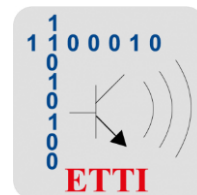




Universitatea POLITEHNICA din București

Facultatea de Electronică Telecomunicații și Tehnologia Informației



Proiect Informatică Aplicată

Tema 14 – Morse Code Player

Studenti:

VLAD Gabriel-Lucian

PASCU Mihai-Adrian

Grupa 411G

An universitar: 2023 - 2024

Cuprins:

I. <u>Descriere Proiect</u>	3
II. <u>Descriere Hardware</u>	4
III. <u>Descriere Software</u>	8
IV. <u>Implementare</u>	8
VII. <u>Bibliografie</u>	22

I. Descriere Proiect:

Lucrarea cu numărul 14 presupune realizarea unui “encoder” de cod morse folosind mai multe tipuri de dispozitive electronice de output (buzzer, LED, LCD). Codul morse este greu de replicat la perfecție de către o persoană reală, așa că proiectarea unui dispozitiv ce poate traduce din limbajul uzual în codul morse poate deveni foarte util datorită posibilității aproape nule de eroare.

Scopul acestei lucrări este de a oferi o interfață ușor de înțeles și utilizată pentru a transmite mesaje codate în morse. De asemenea, lucrarea are ca scop familiarizarea cu device-uri electronice de bază (microcontroller, LCD, buzzer).

Principiul de funcționare este destul de simplu: user-ul va apăsa la Serial Monitor șirul de caractere ce dorește să îl transmită în cod morse, iar output-ul va fi unul vizual (LCD+LED), cât și auditiv (buzzer). Toate cele 3 componente de output funcționează în sincron, în timp ce sunetul va fi redat de buzzer, LED-ul va sta aprins, iar pe LCD vor fi afișate atât caracterele ce sunt transformate în cod morse, cât și punctele sau liniile ce alctuiesc respectivele caractere.

Codul morse este un limbaj universal, ce poate fi înțeles de orice persoană, indiferent de limba pe care o vorbește. Fiind un limbaj universal există și reguli stricte în ceea ce privește comunicarea folosind codul morse. Fiecare caracter corespunde unui șir de puncte/linii (dit/dah). Liniile au durată o unitate de timp în morse, în timp ce liniile au o durată de trei unități de timp în morse. De asemenea, există și reguli și pentru spațierea dintre caractere și cuvinte, între fiecare caracter trebuie să existe o pauză de 3 unități de timp, iar între fiecare cuvânt pauza va fi de 7 unități de timp.

A a	• —	J j	• — — —	S s	• • •
B b	— • • •	K k	— • —	T t	— —
C c	— • — •	L l	• — • •	U u	• • —
D d	— • •	M m	— —	V v	• • • —
E e	•	N n	— •	W w	• — —
F f	• • — •	O o	— — —	X x	— • • —
G g	— — •	P p	• — — •	Y y	— • — —
H h	• • • •	Q q	— — • —	Z z	— — • •
I i	• •	R r	• — •		

(Figura 1.1)

Folosindu-ne de urmatorul tabel putem identifica fiecare litera cu un anumit sir de puncte/linii ce corespund pentru acea litera in codul morse.

II. Descriere hardware:

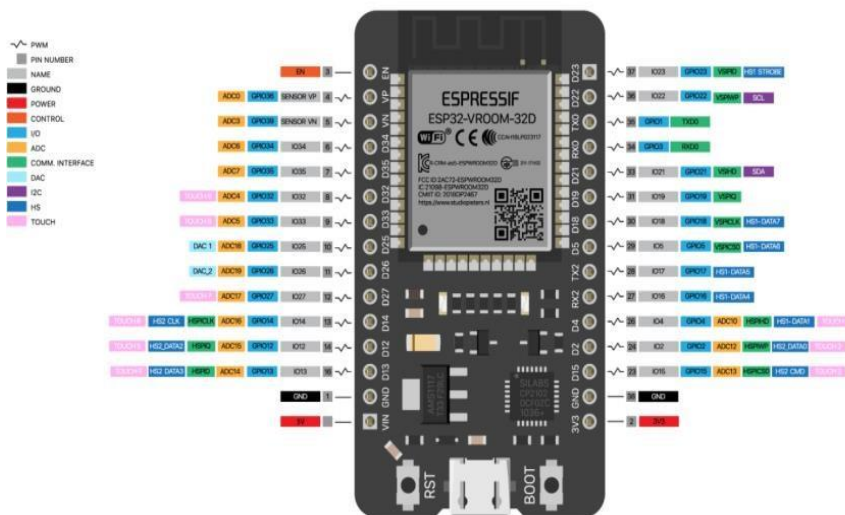
Pentru a putea realiza proiectul am utilizat urmatoarele materiale:

1. Modul ESP32 bazat pe microprocesor Tensilica Xtensa LX6
2. Breadboard Full-Size (x2)
3. Fire de legatura
4. Buzzer piezoelectric
5. LED alb
6. Ecran LCD 16x2
7. Potentiometru
8. Rezistori 220 Ohm (x2)

1. Modulul ESP32

Modulul ESP32 bazat pe microprocesorul Tensilica Xtensa LX6, fabricat de Espressif Systems, dispune de doua nuclee ce pot opera la frecvente intre 160 si 240MHz. De asemenea modulul ESP32 are integrat si un system WiFi si Bluetooth (classic sau BLE).

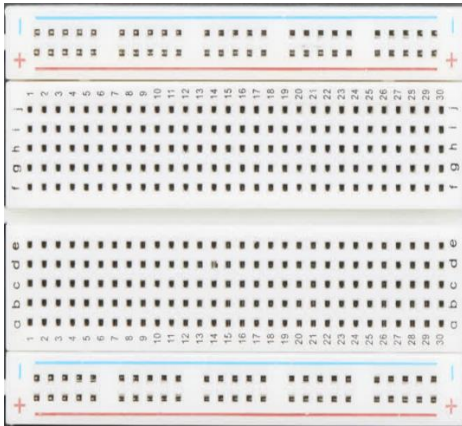
Modulul dispune de un numar mare de pini, astfel ii confera o aplicabilitate foarte mare. Numarul pinilor poate varia in functie de producator. In cadrul acestui proiect am folosit modelul ESP32-WROOM-32D ce dispune de 38 de pini numerotati in functie de denumirea interna a pinilor (GPIOxx).



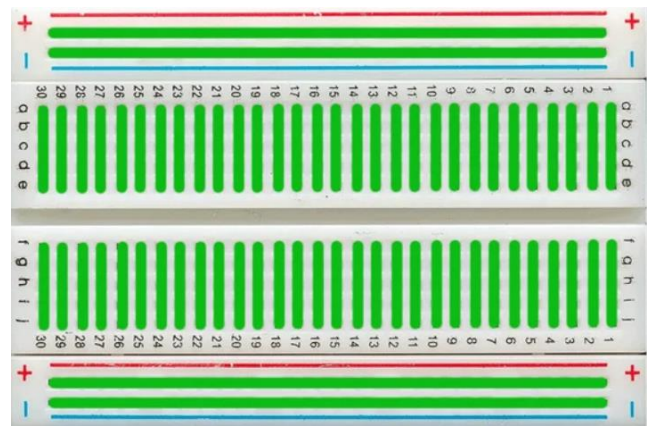
(Figura 2.1)

2. Breadboard

Breadboard-ul (sau placa de prototipare) este foarte des folosit in proiectarea circuitelor, mai ales in fazele incipiente ale unui proiect. Este foarte popular in mediul educational, datorita flexibilitatii sale oferite de faptul ca circuitile nu au nevoie de niciun fel de lipiri pentru a functiona. Placa de prototipare este alcatuita din mici perforatii in interior carora se afla cleme dintr-un material cu rezistenta foarte mica (conductor) ce sunt legate intre ele dupa cum se poate observa in figura 2.3:



(Figura 2.2)



(Figura 2.3)

3. Fire de legatura

Firele de legatura utilizate in prezenta lucrare sunt fire metalice, izolate, cu o impedanta mica, astfel influenta lor asupra circuitului este una minima, neglijabila.

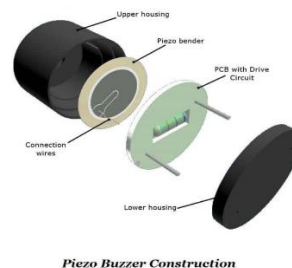
4. Buzzer piezoelectric

Buzzer-ul piezoelectric (fig. 2.4) este o componenta activa ce se foloseste de un semnal dreptunghiular alternativ la o anumita frecventa (in functie de materialul piezoelectric utilizat) pentru a produce sunete. La primirea unui semnal electric, materialul piezoelectric vibreaza (fig 2.5), in urma acestei oscilatii este produs un sunet.

In acest proiect am utilizat buzzer-ul pentru a asigura output-ul auditiv al dispozitivului. Buzzer-ul genereaza sunete la o anumita frecventa setata in codul sursa, sunetele respectand regulile de baza ale codului morse (unitatea de timp).



(Figura 2.4)



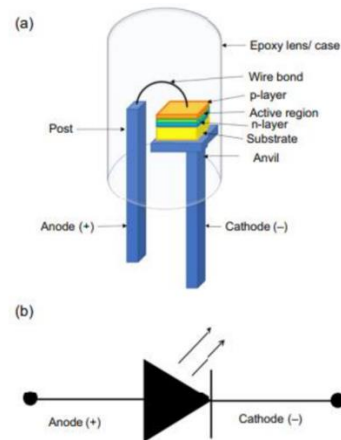
(Figura 2.5)

5. LED

LED-ul (Light Emitting Diode) este o dioda semiconductoră ce emite lumina la polarizarea directă a jonctiunii p-n. LED-ul este o componentă sensibilă ce necesită un flux electric constant pentru a putea funcționa în parametrii normali. Spre deosebire de lampile incandescente obișnuite, LED-ul poate emite lumina doar în cazul în care curentul este aplicat de la anodul spre catodul diodei. În prezentul proiect, LED-ul este utilizat ca dispozitiv de output, acesta emitând lumina odată cu sunetul emis de buzzer.



(Figura 2.6)

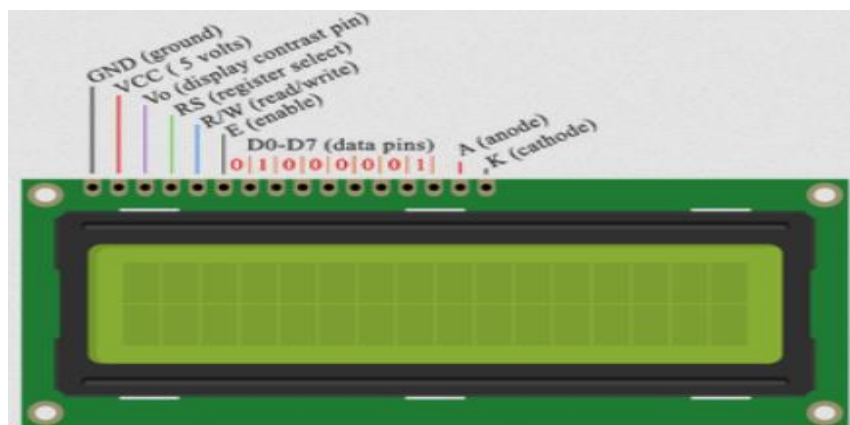


(Figura 2.7)

6. Ecran LCD

LCD-ul (Liquid Crystal Display) este un dispozitiv de afișare pentru caractere, cifre, litere sau imagini. LCD-ul este construit dintr-o matrice de celule lichide care devin opace sau își schimbă culoarea sub influența unui curent electric.

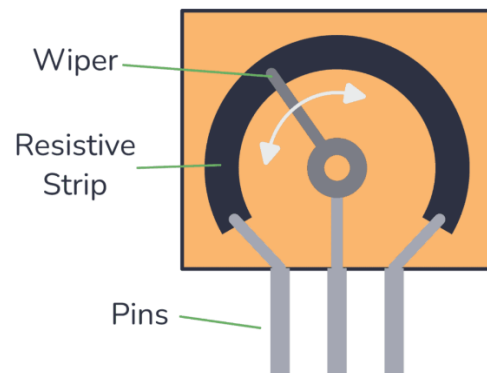
În acest proiect am utilizat un LCD cu 16 coloane și 2 linii ce facilitează output-ul liniilor și al punctelor în timpul redării codului morse. Pentru a putea utiliza LCD-ul în acest scop am făcut legăturile necesare la modulul ESP32 și în codul pe care acesta îl execută.



(Figura 2.8)

7. Potentiometru

Potentiometrul este o componenta cu rezistență variabilă, ce poate fi modificată de către utilizator în mod analog. În acest proiect am utilizat potentiometrul pentru a varia contrastul LCD-ului.



(Figura 2.9)

8. Rezistor

În acest proiect am utilizat 2 rezistențe cu valoarea de 220 de Ohmi pentru a proteja LED-ul alb și LED-urile ecranului LCD pentru a nu se arde.



(Figura 2.10)

III. Descriere Software:

Partea de software a proiectului este asigurată de programul “Arduino IDE” ce ne oferă infrastructura necesară pentru a proiecta codul sursă pentru această temă. Modulul ESP32 este conectat printr-un cablu USB-A - Micro-USB la un computer ce poate rula “Arduino IDE”.

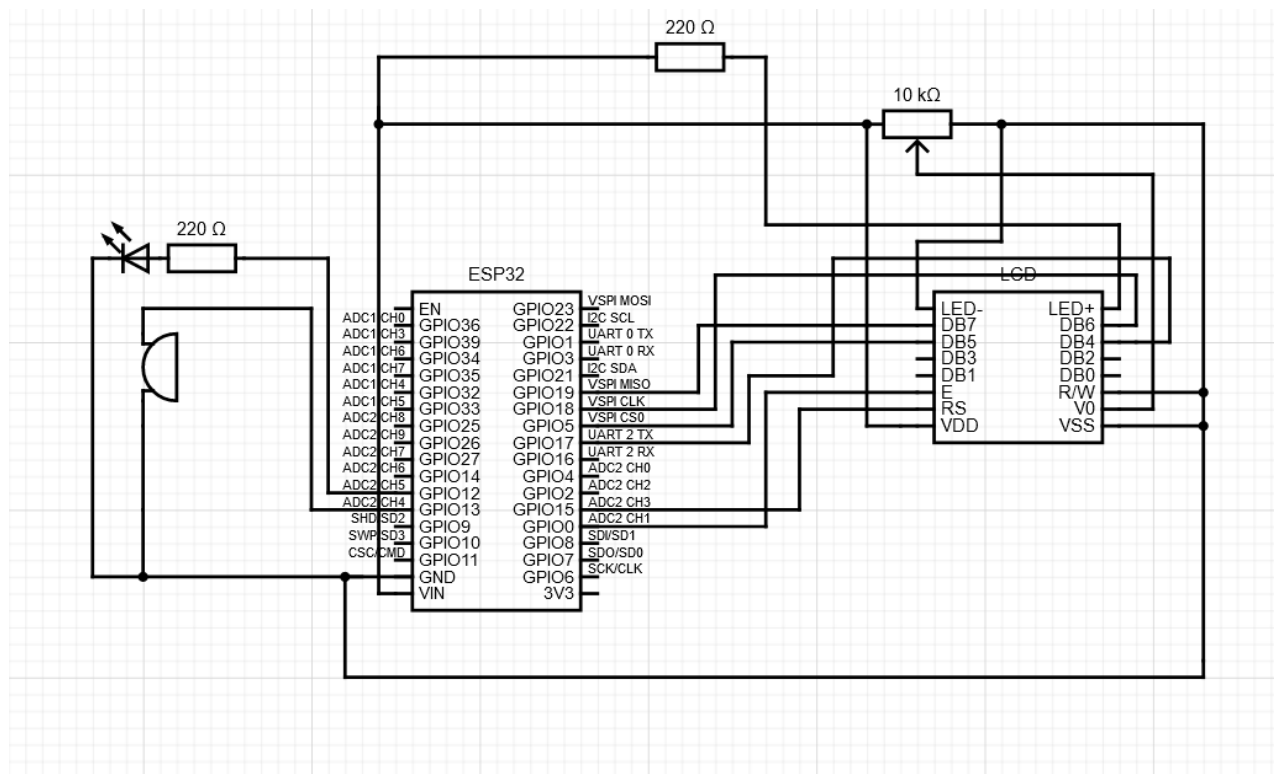
Arduino IDE are preinstalată atât librăria Arduino.h, cât și librăria “ESP32 analogWrite”. La configurarea mediului de programare. De asemenea a fost nevoie să adăugăm următoarele două librării: “ctype.h” pentru a putea folosi funcția toupper(); am adăugat și librăria “LiquidCrystal.h” pentru a putea utiliza ecranul LCD.

În cadrul mediului de programare “Arduino IDE” se poate accesa și “Serial Monitor” pentru a putea citi sau a trimite date către modulul ESP32.

IV. Implementare:

a) Implementare Hardware

Pentru a conecta elementele circuitului la modulul ESP32 am realizat montajul necesar pe placa de prototipare, urmând următoarea diagramă:

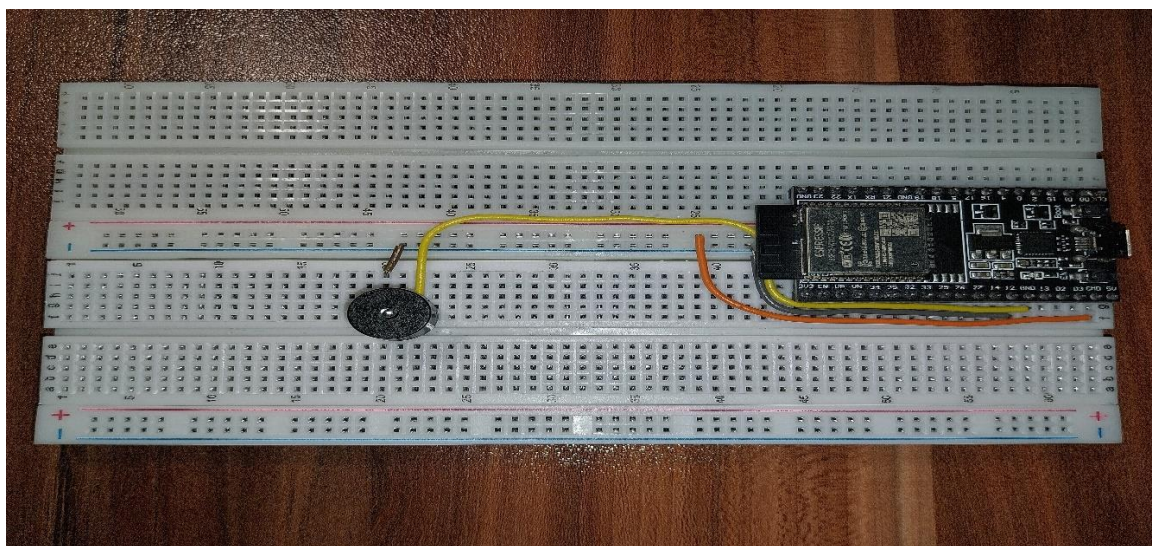


(Figura 4.1)

Conform diagramei am facut urmatoarele conexiuni:

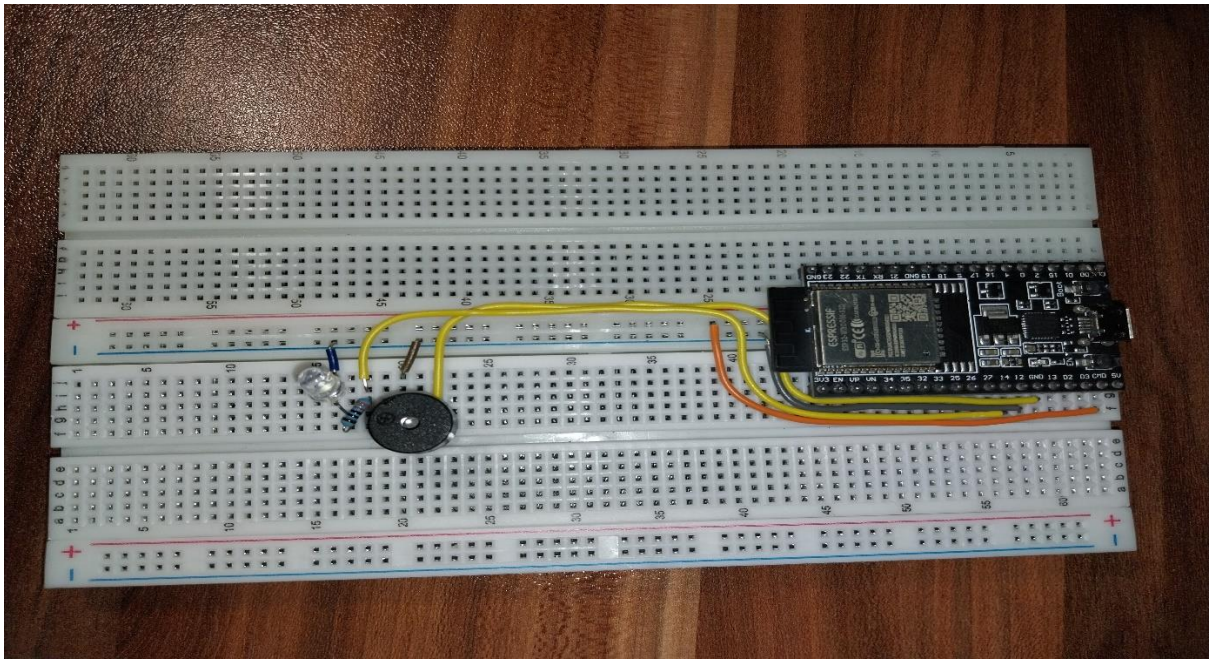
1. Pinul **GPIO13** la Buzzer – alimenteaza Buzzer-ul pentru a genera sunete
2. Pinul **GPIO12** la grupul rezistenta+LED – alimenteaza LED-ul pentru a se aprinde la semnalele morse
3. Pinul **GPIO0** la pinul **Enable** al LCD-ului - permite scrierea in registrele de memorie ale LCD-ului
4. Pinul **GPIO15** la pinul **Register Select (RS)** al LCD-ului – controleaza registrele care sunt modificate
5. Pinul **GPIO17** la pinul **D4** al LCD-ului – transmite date la LCD
6. Pinul **GPIO5** la pinul **D5** al LCD-ului – transmite date la LCD
7. Pinul **GPIO18** la pinul **D6** al LCD-ului - transmite date la LCD
8. Pinul **GPIO19** la pinul **D7** al LCD-ului - transmite date la LCD
9. Pinul **LED-** al LCD-ului la **GND** – masa backlight-ului pentru LCD
10. Pinul **LED+** al LCD-ului la potentialul de **5V**, trecand printr-o rezistenta de 220 Ohm – potentialul pentru backlight-ul LCD-ului
11. Pinul **VDD** al LCD-ului la potentialul de **5V**
12. Pinul **VSS** al LCD-ului la **GND**
13. Pinul **V0** al LCD-ului la potentialul de **5V**, trecand prin rezistenta variabila de 10kOhm – potentiometrul are rolul de a ne ajuta sa ajustam contrastul LCD-ului

Primul pas in proiectarea circuitului pe breadboard a fost fixarea modului ESP32 in cele doua breadboard-uri pentru a putea avea access la toti pinii de care modulul dispune. Astfel, dupa ce ne-am asigurat ca modulul este fixat am conectat liniile de 5V si de GND de pe breadboard-uri la ESP32, urmand sa conectam **Buzzer-ul** la pinul **GPIO13**:



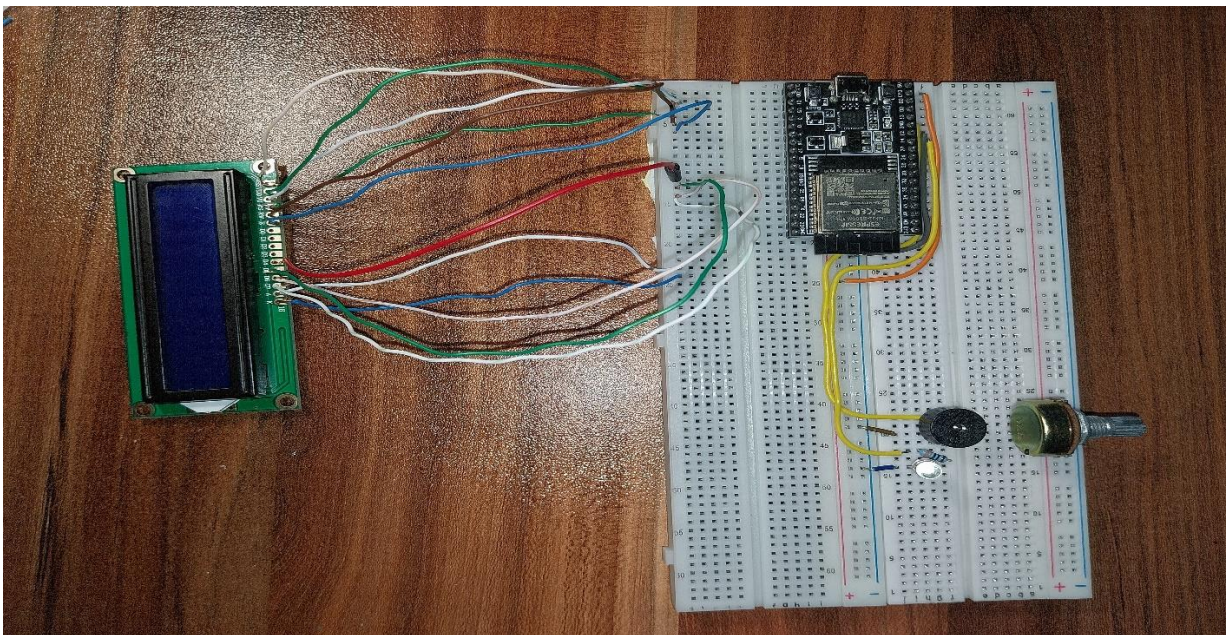
(Figura 4.2)

Dupa conectarea Buzzer-ului a urmat conectarea **LED-ului** la pinul **GPIO12**. De asemenea, in serie cu LED-ul am legat si o rezistanta de 220 de Ohmi pentru a proteja LED-ul:



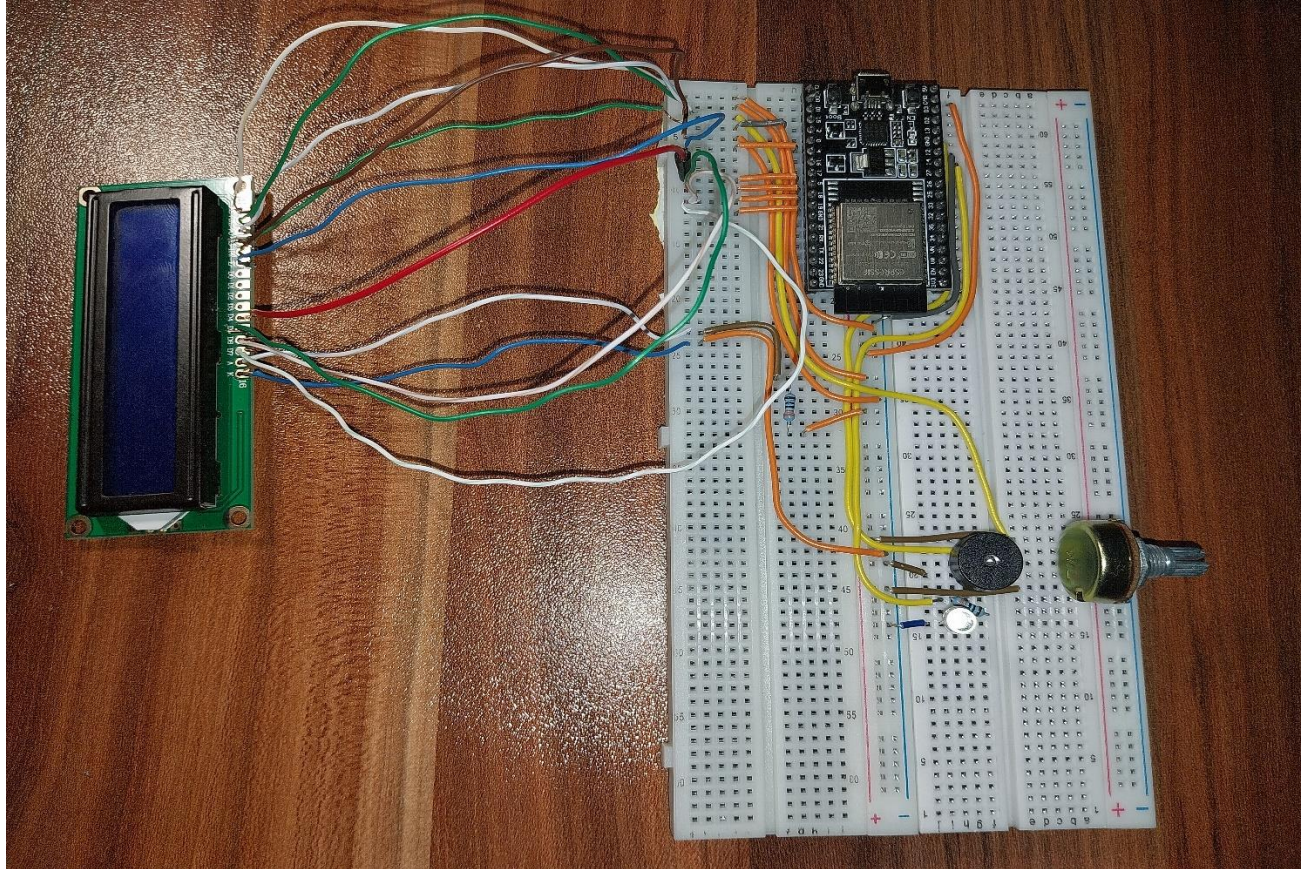
(Figura 4.3)

Urmatorul pas a fost legarea LCD-ului la breadboard pentru a putea face conexiunile necesare pentru acesta. De asemenea, am pus si potentiometrul pe placa de prototipare:



(Figura 4.4)

În final am făcut și restul conexiunilor (de la fiecare pin necesar pentru funcționarea LCD-ului la pinii de pe modulul ESP32). Astfel montajul nostru este gata, iar acum putem începe să implementăm partea de software



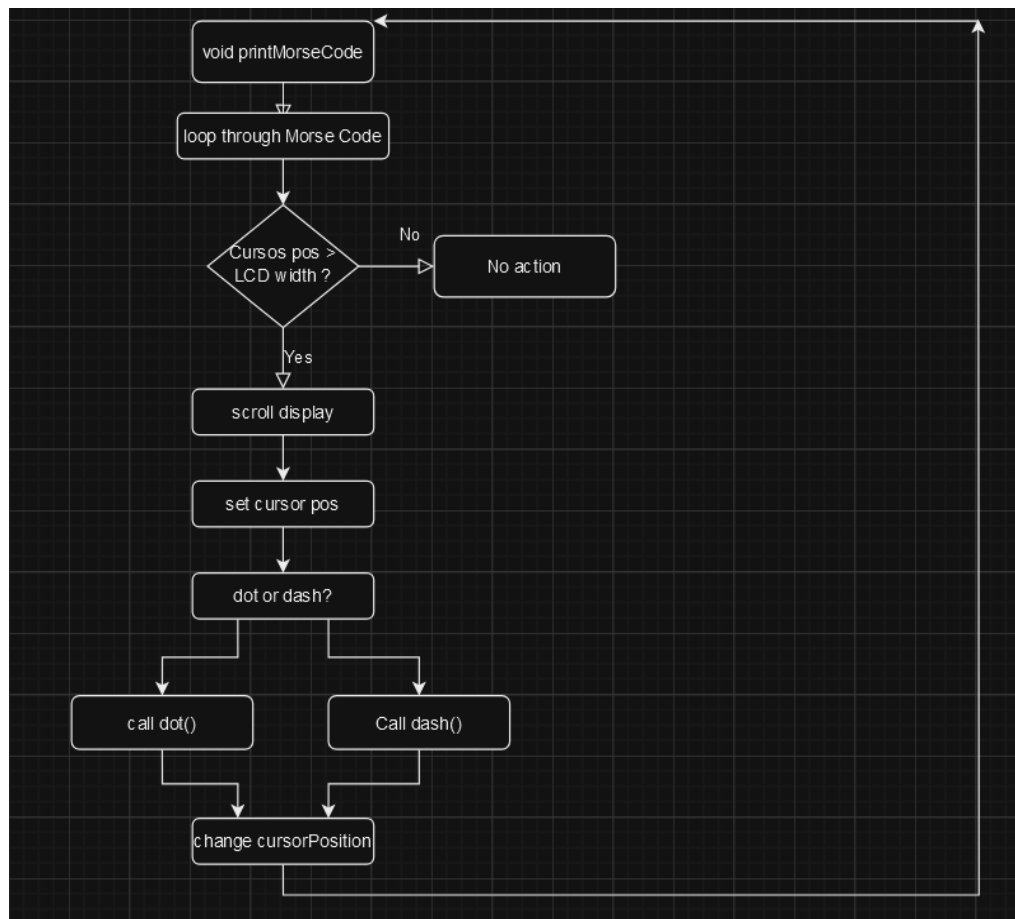
(Figura 4.4)

b) Implementare Software

Pentru a putea duce la final proiectul nostru mai trebuie să proiectăm partea de software pe care o vom scrie în mediul de programare “Arduino IDE”, iar apoi vom transmite codul modulului ESP32 ca acesta să îl poată executa. Pentru implementarea software-ului am dezvoltat o serie de funcții ce ne ajută să obținem output-ul dorit: redarea codului morse pe dispozitivele electronice utilizate.

- Functia void printMorseCode()

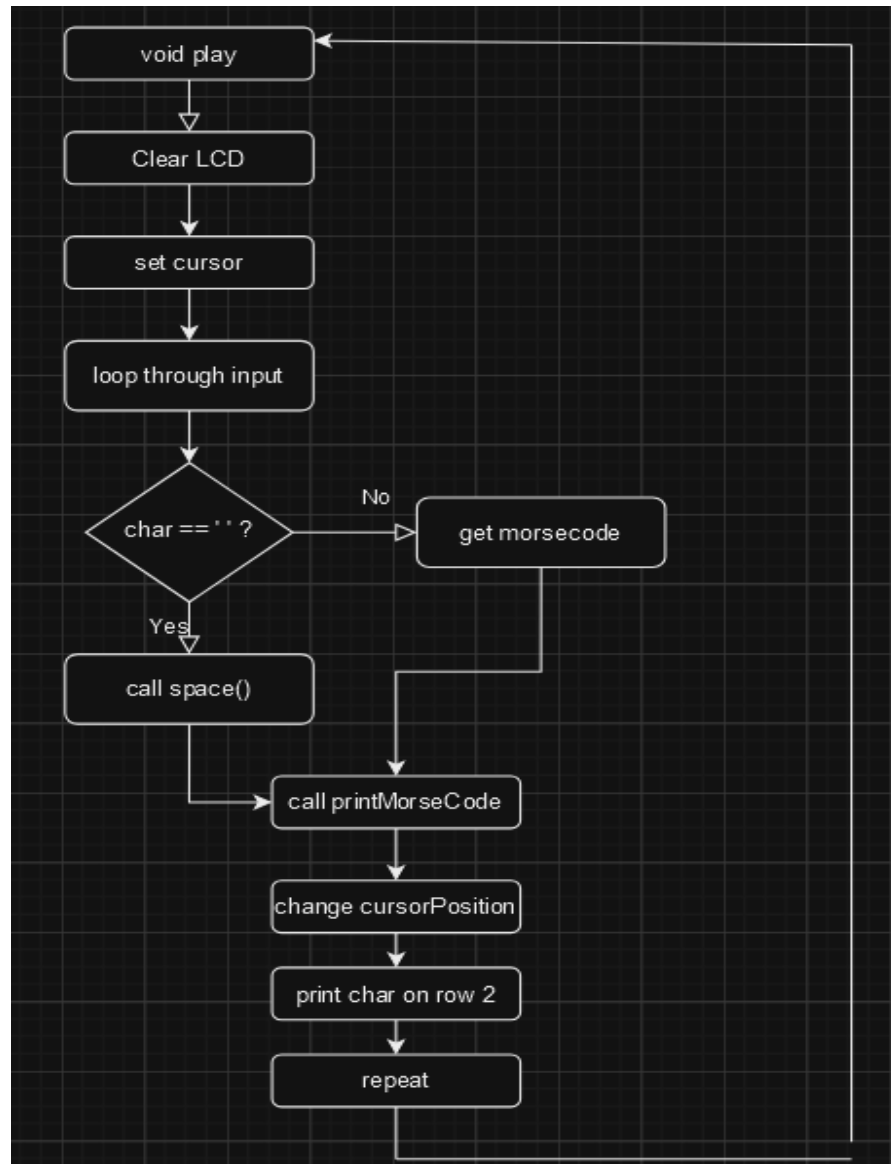
Aceasta functie este esentiala pentru program, deoarece executa toate comenzile legate de output. In aceasta functie se verifica caracterul citit din array-ul "codeset[]" si se comanda output-ul in functie de acest caracter.



(Figura 4.5)

- Functia void play()

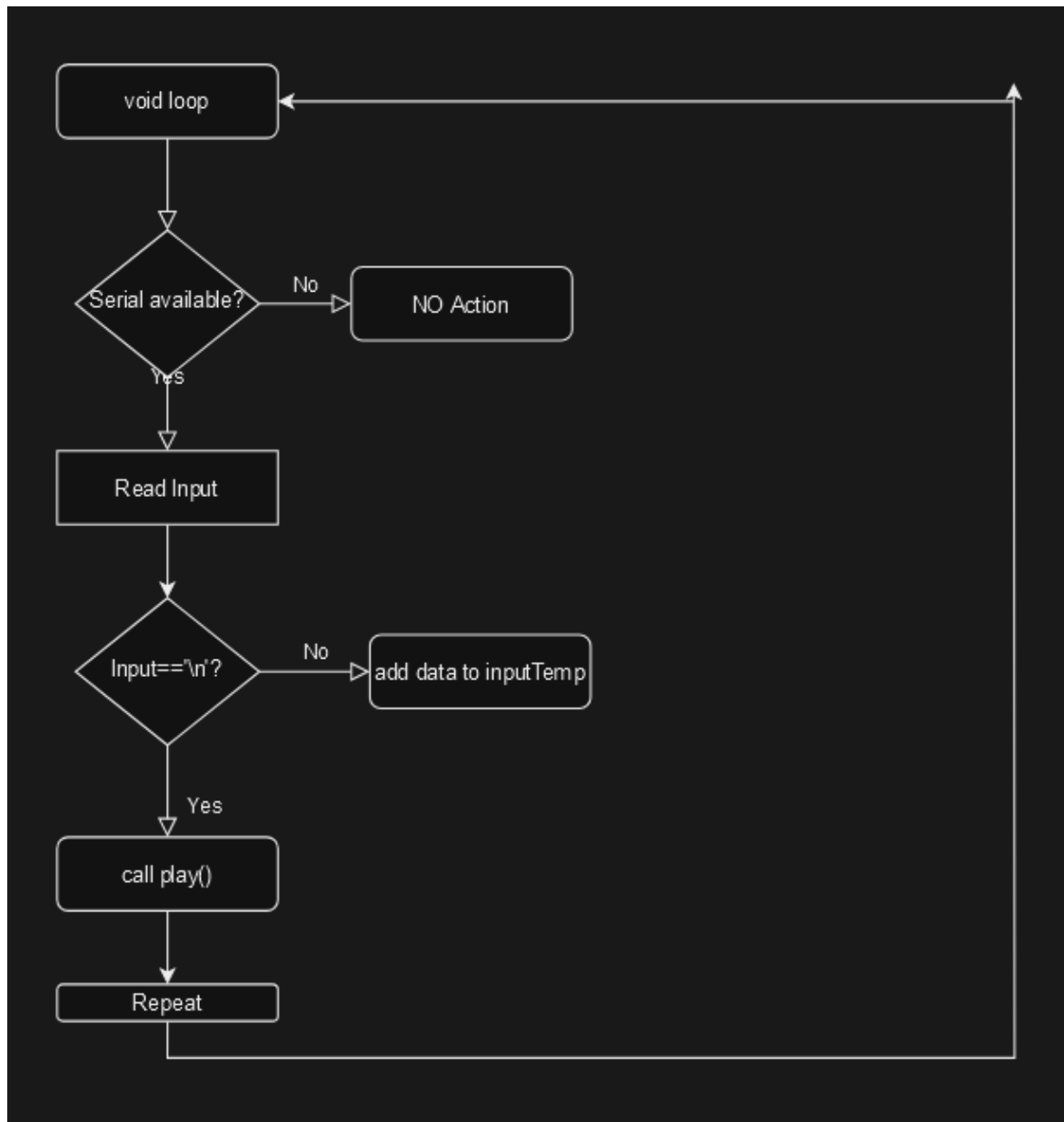
Aceasta functie este esentiala pentru program, deoarece executa toate comenzile legate de output. In aceasta functie se verifica caracterul citit din array-ul "codeset[]" si se comanda output-ul in functie de acest caracter.



(Figura 4.6)

- Functia void loop()

Functia “void loop()” verifica daca exista date disponibile in input-ul temporar si citeste caracterele pe rand. Daca detecteaza sfarsitul unui string (“\n”), transforma toate caracterele acumulate in input-ul temporar in cod morse si le afiseaza pe LCD si in Serial Monitor.



(Figura 4.7)

```
#include <ctype.h>

#include
<LiquidCrystal.h>

const int rs = 15, en =
0, d4 = 17, d5 = 5, d6
= 18, d7 = 19;

LiquidCrystal lcd(rs,
en, d4, d5, d6, d7);

int LED_pin = 12;

int BUZZER_pin = 13;

const int unitLength =
100;

const int dotLength =
unitLength;

const int dashLength =
unitLength * 3;

const int charLength =
unitLength * 3;

const int wordGapLength
= unitLength * 7;

int pitch = 1000;

const char* codeset[] =
{
/* A */ ".-",
/* B */ "-...",
/* C */ "-.-.",
/* D */ "-..",
```

```
/* E */ ".",
/* F */ "..-",
/* G */ "--.",
/* H */ "....",
/* I */ "..",
/* J */ ".---",
/* K */ "-.-",
/* L */ "-..",
/* M */ "--",
/* N */ "-.",
/* O */ "---",
/* P */ ".--.",
/* Q */ "--.-",
/* R */ "-.",
/* S */ "...",
/* T */ "-",
/* U */ "..-",
/* V */ "...-",
/* W */ "-.-",
/* X */ "-..-",
/* Y */ "-.--",
/* Z */ "--.."
};

const char*
getCode(char c) {
    c = toupper(c);
    if ('A' <= c && c <=
        'Z') {
```



```
return codeset[c -
'A'];
}
return 0;
}

void setup() {
  lcd.begin(16, 2);
  pinMode(LED_pin,
  OUTPUT);
  pinMode(BUZZER_pin,
  OUTPUT);
  Serial.begin(9600);
}

void printDotDash(char
symbol) {
  digitalWrite(LED_pin,
  HIGH);
  lcd.print(symbol);
  Serial.print(symbol);

  if (symbol == '.') {
    tone(BUZZER_pin, pitch,
    dotLength);
    delay(dotLength);
  } else if (symbol == '-'
  ') {
```

```
tone(BUZZER_pin, pitch,  
dashLength);  
delay(dashLength);  
}
```

```
digitalWrite(LED_pin,  
LOW);  
delay(unitLength);  
}
```

```
void dot() {  
printDotDash('.');  
}
```

```
void dash() {  
printDotDash('-');  
}
```

```
void letter() {  
Serial.print(" ");  
delay(charLength);  
}
```

```
void space() {  
lcd.print(" ");  
Serial.println();  
delay(wordGapLength);  
}
```

```
void  
printMorseCode(const  
char* code, int  
&cursorPos) {  
for (int j = 0; code[j]  
!= '\0'; j++) {  
if (cursorPos >= 16) {  
lcd.scrollDisplayLeft()  
;  
}  
lcd.setCursor(cursorPos  
, 0);  
if (code[j] == '.') {  
dot();  
if (cursorPos >= 16) {  
lcd.scrollDisplayLeft()  
;  
}  
} else if (code[j] ==  
'-') {  
dash();  
if (cursorPos >= 16) {  
lcd.scrollDisplayLeft()  
;  
}  
}  
cursorPos++;  
}  
}
```

```
void play(const char*
input) {
int inputLength =
strlen(input);
lcd.clear();
int cursorPos = 0;

for (int i = 0; i <
inputLength; i++) {
char c = input[i];

if (c == ' ') {
space();
cursorPos+=2;
} else {
const char* code =
getCode(c);
printMorseCode(code,
cursorPos);
letter();
cursorPos+=1;
lcd.setCursor(i, 1);
lcd.print(c);
}
}
}

char inputTemp[255];
```

```
int tempIndex = 0;

void loop() {
  while
  (Serial.available() >
  0) {
    char inputText =
    Serial.read();
    if (inputText == '\n')
    {
      inputTemp[tempIndex] =
      '\0';
      play(inputTemp);
      tempIndex = 0;
      delay(500);
    } else {
      if (tempIndex <
      sizeof(inputTemp) - 1)
      {
        inputTemp[tempIndex++]
        = inputText;
      }
    }
  }
}
```


VII. Bibliografie:

- Buzzer: <https://bgsu.instructure.com/courses/1157282/pages/tutorial-active-buzzer>, <https://nerdytechy.com/active-vs-passive-buzzer/>
- Mediu de programare: <https://www.arduino.cc/en/software>
- Lucrarile de laborator postate pe Microsoft Teams
- Conectarea LCD-ului la ESP32: <https://docs.arduino.cc/learn/electronics/lcd-displays/>
- Guideline-uri pentru software: <https://www.arduino.cc/education/morse-code-project/>
- SOURCE CODE: <https://github.com/VladGabrielLucian/PIA-Project/tree/main>